Sprint 3 — Scenario-specific Test Data: Gate 0 Findings

Defect reproduced ✅

Regenerated Add Employee suite (`SPRINT2_ADD_EMPLOYEE_OUTPUT.md`) shows pervasive generic placeholders:
- **Every positive scenario**: “Enter a valid First Name”, “Enter a valid Middle Name”, “Enter a valid Last Name”
- **Negative format scenario**: “Enter an invalid First Name” (no specificity — what makes it invalid?)
- **No business realism**: no actual names, no boundary values tied to real field constraints

Evidence count: 50+ occurrences of “valid/invalid First Name” across 29 scenarios. Zero scenario-specific values.

Root cause 📍

File: `src/engines/scenario-builder.ts` , function `dataPhraseFor()` (lines 506-550)

What it does correctly

- ✅ Intent-based dispatch for **negative/security** scenarios works:
- SQL injection → `"' OR 1=1 --"`
- XSS → `"<script>alert(1)</script>"`
- Duplicate → `"a First Name that already exists"`
- Whitespace-only → `"  "`
- File invalid format → `"virus.exe"`

What falls through to generic

- ❌ **Positive scenarios** (coverageType = `'positive'`) → line 548: `return "a valid ${label}"`
- ❌ **Boundary scenarios** without max/min in intent → `"a boundary-length ${label}"`

Why it’s generic

The `FieldLike` interface (line 114) captures **NO field constraints**:

```
interface FieldLike {
  name?: string;
  type?: string;
  required?: boolean;
  selector?: string;
  label?: string;
  // ❌ NO: minLength, maxLength, pattern, unique, sampleValues, format
}
```

So `dataPhraseFor()` has:
- ✅ The scenario intent (title, objective, id, riskArea)
- ✅ The field identity (name, label, type)

- ❌ **NO field constraints** to generate realistic boundary values
- ❌ **NO sample valid values** for positive cases

Result: falls back to “a valid First Name” because it has nothing else to ground in.

The Fix (scope for Sprint 3)

1. Enhance field metadata (NO schema change to App Profile — use what’s available)

The requirement-understanding engine already extracts `businessRules` (line 176-194 in `requirement-understanding-engine.ts`):
- Parses length/format/unique/range constraints from requirement text
- Returns unstructured excerpts (e.g., “First Name is mandatory and max 50 characters”)

Action: Pass these constraints through to `dataPhraseFor()` via a new optional `constraints` parameter or enhance `FieldLike` minimally (local to scenario-builder).

2. Update `dataPhraseFor()` to generate scenario-specific values

Positive scenarios:
- Name fields → realistic names: `"Emma Watson"`, `"Raj Kumar"`, `"李明"`
- If maxLength is known → value near the boundary: `"A 48-character name (just under the 50-char limit)"`
- Date fields → realistic dates: `"2024-01-15"`, `"today's date"`
- Email → `"emma.watson@example.com"`
- **NO generic “a valid First Name”**

Boundary scenarios:
- If maxLength known → `"A 51-character name (1 over the 50-character limit)"`
- If minLength known → `"A single-character name (minimum boundary)"`
- **Grounded in actual constraints**, not just “boundary-length”

Negative format:
- Email field + invalid format → `"invalid-email-format"` or `"notanemail"`
- **Field-type-aware**, not just “an invalid First Name”

3. Constraints frozen to requirement text only

What we will NOT do (per your discipline):
- ❌ Invent constraints the requirement doesn’t state
- ❌ Guess field lengths or formats
- ❌ Change the App Profile schema
- ❌ Add LLM calls to generate data
- ❌ Touch the planner, the KB, or any other engine

Fallback when constraints are unknown: prefer a realistic example over “valid” — e.g., `"Emma Watson"` > `"a valid First Name"` even if we don’t know the max length. The human tester can execute “Enter Emma Watson” directly; they cannot execute “Enter a valid First Name”.

Success criteria (v1-complete alignment)

After Sprint 3, regenerate Add Employee and verify:
- ✅ **Generic test data = 0** (no “a valid First Name” anywhere)
- ✅ **Boundary values tied to field metadata** (max/min length assertions reference real constraints when available)
- ✅ **Injection payloads tied to affected fields** (already working — verify no regression)
- ✅ **Duplicate values tied to unique identifiers** (already working — verify no regression)
- ✅ **Business-realistic values** for positive cases (names that look like names, emails that look like emails)
- ✅ **No invented constraints** (if requirement says nothing about max length, we use a realistic example but don’t claim “50 characters” unless it’s stated)

Before/After (expected transformation)

Scenario	Before (Sprint 2)	After (Sprint 3)
Positive (happy path)	“Enter a valid First Name”	“Enter ‘Emma’ in the First Name field”
Positive (happy path)	“Enter a valid Email”	“Enter ‘emma.watson@example.com’ in the Email field”
Boundary (max length)	“Enter a boundary-length First Name”	“Enter a 51-character name (1 over the 50-character limit) in the First Name field” (if req states max 50)
Negative (format)	“Enter an invalid Email”	“Enter ‘not-an-email’ (invalid format) in the Email field”
Duplicate	“Enter a First Name that already exists”	✅ Already correct
SQL injection	“Enter the SQL-injection string”	✅ Already correct

---

Implementation plan

1. ✅ Gate 0 — defect reproduced (this document)
2. **Root cause** — identified `dataPhraseFor()` + missing field constraints
3. **Implement** — enhance `dataPhraseFor()` with realistic value generation:
   - Extract any available constraints from requirement businessRules

- Add field-type-aware realistic examples (name→names, email→emails, date→dates)
- Preserve intent-based dispatch for negative/security (already correct)

4. **Proof** — before/after table showing generic→specific transformation
5. **Regression** — run core test suites (scenario-builder, expected-result-validator, scenario-planner, generation-quality)
6. **Manual inspection** — regenerate Add Employee, audit for zero “valid data” placeholders
7. **PR #318** — commit, push, open for review (NOT merge)

Questions for user before proceeding

1. **Constraints source**: Should I parse `businessRules` excerpts (already extracted by requirement-understanding engine) to find length/format constraints, or is there a cleaner source I’m missing?
2. **Realistic examples without constraints**: When the requirement provides NO constraints (e.g., no max length stated), is it acceptable to use a realistic example like `"Emma Watson"` instead of `"a valid First Name"`, knowing we’re not grounding it in stated limits? (Alternative: keep “a valid First Name” when we have no constraints — but that defeats the “business-realistic” success criterion.)
3. **Freeze confirmation**: Should I implement this **purely inside** `dataPhraseFor()` (no schema changes, no new engines, no planner/KB edits)? That matches your “nothing else” instruction.

Ready to proceed once you confirm the approach.